关键特性（编程模型、调度方式、IO、、、概念

OS前行、思考题

# 物联网 OS

## ⭐ 物联网OS概念

- **硬件抽象，释放硬件能力**：把复杂的硬件操作封装成统一接口
- **资源管理，提升全局能效**：统一分配CPU、内存等资源，安排任务和休眠，减少耗电。
- **使能应用，提供生态入口**：提供网络协议、开发工具和软件库，方便开发应用、连接云平台。

物联网操作系统发展现状：数量众多、百家争鸣

## ⭐ 与传统OS区别

1. **存储空间占用不同**：物联网OS内存占用较小，一般只有几十KB
2. **内存管理方式不同**：有MMU vs. 无MMU
3. **OS特权模式不同**：很多物联网芯片不具有CPU特权模式，所有的指令都能访问任何硬件资源

有Bug的程序可以在不受操作系统的控制下轻松地占用CPU周期

## 概述：系统构成

| | | |
|---|---|---|
| **嵌入式OS相关组件** | **任务调度** | **两种主要的任务调度方式：事件驱动任务调度、多线程任务调度** |
| | **动态加载** | **允许模块动态加载，对物联网应用软件更新非常有用** |
| | **存储空间管理** | **数据内存（随机存储器RAM）、程序闪存（Flash）** |
| | **资源抽象** | **专用资源抽象、虚拟化资源抽象和共享资源抽象** |
| | **传感器接口** | **为获取传感器读数提供了方便统一的方法** |
| **无线网络相关组件** | **网络协议栈** | **处理底层细节：无线芯片配置、介质访问控制和队列管理；支持高级网络服务：TCP连接、数据收集与分发处理底层细节** |
| **AI支持相关组件** | **TinyML** | **TensorFlowLite，Split Computing功能组件** |

**两个重要概念**

1. **Library OS**（库操作系统）：将OS的某些功能作为**库**而不是一个单独的内核运行
   a. 允许应用程序**直接调用**OS服务
   b. 可以**提高性能**，因为减少了用户态和内核态之间的上下文切换，并且可以**为特定应用程序定制操作系统功能**
2. **微内核**（Microkernel）：提供了OS最基本的核心功能，如任务/线程管理、内存管理和进程间通信（IPC）。
   a. 其他服务如文件系统、设备驱动和网络协议等，通常作为**用户态**的进程运行，并通过**消息传递**与微内核通信。
   b. 使得操作系统具有高度的**模块化和灵活性**，因为服务和驱动可以独立地添加、更新或替换，而不需要对整个内核进行修改。

c. ***宏内核***（Monolithic Kernel）：将操作系统的许多功能集成在一个大型的内核态进程中。例如Linux。

# ⭐ 关键特性

## 1 编程模型

1. **依据`程序结构`分类**：
   a. ***非模块化编程***：应用程序作为一个**整体**来进行设计、测试和调试
   b. ***模块化编程***：将程序的功能分离为独立的、可互换的模块，每个模块执行某一具体功能
      i. 编码层面的模块化，如**TinyOS**组件化编程（编译后不再保留组件化信息）
      ii. 二进制层面的模块化，如**SOS**模块化编程（编译后仍有模块信息、可动态加载）
2. **依据`执行逻辑`分类**：
   a. ***基于事件驱动的编程***：应用程序会**实时地**处理或响应系统中的事件；应用程序由**多个事件处理函数**构成（实时性和响应性更好）
   b. ***基于多线程的编程***：一个线程对应一个逻辑上的任务（减少**代码分片**带来的代码量）

## 2 `调度方式`

抢占式、非抢占式 lock、协作式（线程主动交出`yield()`）、时间片轮转 lock

**思考题1**

给定三个线程a、b、c，它们单独执行完的时间分别为10s、20s、30s

1. 非抢占式：时间点0s时，a,b,c顺序启动，三个线程完成时间分别是多少？10、30、60
2. 抢占式：线程的优先级为c>b>a（优先级高会抢占优先级低的线程），线程a、b、c的就绪时间点分别为第1s、第2s、第15s，三个线程的完成时间为多少？61、52、45
3. 时间片轮转：时间片的长度为10s，线程a、b、c的启动时间点分别为第0s、第10s、第20s，三个线程的完成时间为多少？10、40、60

## 3 I/O 操作方式

1. **阻塞式I/O**：当程序调用一个I/O操作时，**直到该I/O操作完成**，这个调用才会返回；**易于编程**
2. **分阶段I/O**：把I/O操作的**调用**和I/O操作的**完成**分成了**两个独立的阶段**
   a. **代码冗长**，但可以保持系统的**高响应性**
   b. **更灵活**，**更好控制并发**
3. **从数据采集角度看**：
   a. **单次采集**：需要时需要时请求一次数据，每次操作的**数据量通常较小**
   b. **周期性采集**：按固定周期获取数据，通常由**定时器或数据准备就绪事件**触发
   c. **高频连续采集**：设备持续产生数据，通常需要**硬件FIFO暂存数据**，驱动通过缓冲机制持续接收；高吞吐场景常需要硬件DMA模块

## 4 内存分配

1. **静态内存分配**（如**TinyOS**）
   a. 静态内存区域在编译时确定，通常由未初始化数据段(.bss)和初始化数据段(.data)构成
   b. 可以通过静态数组的方式来实现
   c. **优点**：效率高、内存池中**无碎片**；**缺点**：不能按需申请

2. **动态内存分配（如SOS）**
    a. 动态内存区域在运行时确定；通过malloc申请，通过free释放
    b. **优点**：可按需分配；**缺点**：容易出现碎片、可能造成安全隐患

## 5 软件更新

1. **更新方式**：有线/无线
2. **BootLoader**是设备在加电后执行的第一段代码（硬件初始化->OS装载到内存->跳转到OS并运行）
3. **具体步骤**：OS启动之前，Bootloader查找升级文件-->擦除flash旧文件-->写入flash-->检查并重启
4. **不同更新方法**：
    a. **基于整个镜像**：所有部分更新
    b. **基于模块化**：软件镜像细分为各个模块，根据需求选择某些模块更新
    c. **基于差分**：将旧版本与新版本的代码做差分，得到升级部分的补丁即**差分文件**（可以对整个镜像/模块进行差分）

**差分**

1. **两种指令**
    - **COPY n old_addr**：从旧文件地址addr开始，复制n字节；**1+2+2=5字节**
    - **ADD n〈新数据〉**：直接加入n字节新数据；**1+2+n=3+n字节**
2. **影响差分文件大小**
    - **差分算法**：能否找到更省空间的ADD、COPY组合。*e.g.* **bsdiff、xdelta3、HDiffPatch**
    - **相似性保持技术**：在不改变功能的前提下，让新旧程序的二进制内容尽量相似。

**思考题2：假设：**

- 旧文件： `A B C D E A C B C`
- 新文件： `W A B C D E A C B C X D E A C B`

1. **那么差分文件大小为多少？具体指令是？**
    a. 方案一： `ADD 1 <W>, COPY 9 0, ADD 1 <X>, COPY 5 3` → 4+5+4+5=**18字节**
    b. 方案二： `ADD 1 <W>, COPY 9 0, ADD 6 <X D E A C B>` → 4+5+9=**18字节**
2. **基于差分的编码方式，在程序头部插入函数和在程序尾部插入函数，哪个差分升级包占用空间比较大？**
    一般是**在程序头部插入函数，差分升级包更大**。因为头部插入会使后续函数地址发生变化，导致相关跳转、调用指令也要修改；尾部插入通常不影响原有函数地址，新旧程序相似度更高，差分包更小。

## 6 网络服务

- **传统传感网**：主要关注**多跳自组织**，多基于 **IEEE 802.15.4**；典型的传感网网络协议包括数据收集协议，数据分发协议，时间同步协议等
- **物联网**：协议更多样，如BLE、ZigBee、Wi-Fi HaLow、LoRaWAN、NB-IoT；重点是**干扰共存、异构互联**

> TinyNet的核心：2.5层抽象、uniMAC统一接口
>
> TinyNet关键模块：邻居发现、通信调度

## 7 AI 支持能力

1. **常见开发框架**：

a. **TensorFlow Lite**：嵌入式Linux设备，如树莓派

b. **TensorFlow Lite for Micro**：微控制器，如ESP32

c. **PyTorch Mobile**：Android、iOS设备

2. **TinyML**：（微型机器学习）是机器学习的一个分支，旨在将模型部署到**微控制器和其他低功耗边缘设备**

a. **特点**：低延迟、保护隐私、精度稍低、低功耗、高可靠、内存小

b. **应用案例**：音频视频推理、穿戴计算、健康监测传感器

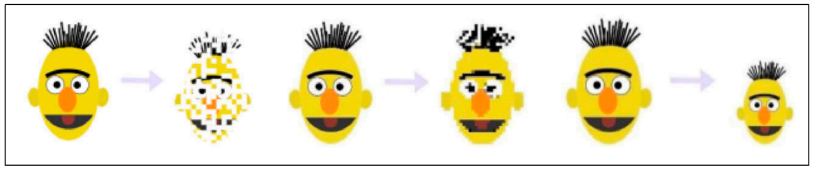
c. **常见优化技术**：

i. **模型压缩**：通过**权重剪枝、量化和知识蒸馏**等方法，减少模型大小

A. **剪枝**：设定**阈值**，删除不重要的权重连接，再**微调**模型

B. **量化**：将**模型参**数从 32 bit 或 64 bit 浮点数变为使用 8 bit 或更低的定点数表示

C. **知识蒸馏**：把一个/多个模型(**教师模型**)学到的知识迁移到另一个**轻量级**模型(**学生模型**)



(a) 剪枝　(b) 量化　(c) 知识蒸馏

ii. **高效架构**：设计紧凑高效的模型结构，如 MobileNetV2

iii. **硬件加速**：充分利用**微控制器**的硬件特性进行优化，如DSP

# ⭐ 各类OS概念、区别

## Contiki OS

一款**开源、高度便携、多任务**的操作系统，适用于**资源有限的无线传感器网络**，专为**具有少量内存的微控制器（MCU）**而设计

**关键特性**

1. **编程模型**：

a. 基于**protothreads**的编程模型，支持**多线程**的编程风格

> Protothreads 编程框架
>
> - **主要特点**：基于**协作式**的调度、用**单个堆栈**实现、不能使用局部变量
> - 以宏PROCESS()开头
>
> 宏AUTOSTART_PROCESSES()用于在程序启动的时候自动地执行当前进程
> 宏PROCESS_THREAD()用来定义protothread的实现
> 宏PROCESS_BEGIN()必须在初始化的变量之后调用
> 宏PROCESS_END()的调用表示当前线程的终止

b. 以**多堆栈**实现的**多线程库mt_thread**：：支持协作式调度（主动yield）和抢占式调度（系统负责抢占）

2. **任务调度**：基于优先级的事件驱动、protothreads调度、基于mt_thread的调度

3. **内存分配**：静态和动态分配

4. **软件更新**：模块化

5. **网络服务**：

a. 路由协议 Contiki **RPL**： 面向**低功耗有损网络**的 **IPv6** 路由协议，支持多种通信模式：多对一（数据收集）、一对多（数据分发）、一对一（点对点）

b. **CTP**：主要支持**多对一数据收集**

c. 轻量级的网络层协议 **uIP**：精简的**TCP/IP协议栈**，减少内存占用

d. IPv6 地址压缩协议 **6LowPAN**：数据包的分割重组、IPv6 包头的压缩和 UDP 包头的压缩

6. **AI支持**：官方不支持，目前开发者自行适配后可使用TensorflowLite
7. **提供基于shell的交互式编程接口**：执行传感数据采集、改变发送能量和改变通信信道等操作；支持编写自定义命令

## RIOT

一个专门面向物联网场景的**实时多线程OS**，基于**微内核**实现，所有的系统服务包括时钟、网络协议栈、网络服务等，都是通过**创建独立的线程**来实现

**特点**：开发者友好、资源占用低（模块化、微内核）、网络支持好（大量网络协议）

### 关键特性

1. **编程模型**：多线程编程（三个Flag字段）
2. **任务调度**：基于优先级的事件驱动
3. **内存分配**：动态分配
4. **软件更新**：模块化（模块在代码目录中表现为**文件夹**形式；几乎所有的**硬件设备**同样被抽象成了模块）
5. **网络服务**：拥有一个**完善的网络协议栈**
   a. 其向下支持大量的**无线**通信协议，向上支持 TCP、 UDP等常用的传输层协议，为设备组网提供了有力的支持
   b. **GNRC协议栈**：为协议栈中的每一层均分配了一个**独立的线程**；底层协议的线程优先级普遍高于高层，例如：MAC层最高，应用层最低。
6. **AI支持**：官方支持，目前兼容TensorflowLite库
7. 提供了**传感器和 SAUL**，可以使用户通过统一的接口分别对**传感器和执行器**进行读取和控制

## HarmonyOS

**面向全场景的分布式操作系统**，适用于多种场景；可以实现单个设备的完整能力，在这个基础之上，HarmonyOS提出了多种终端形式全适配的分布式理念。在同一套系统框架内，支持**1+8+N**等各类终端设备，实现各类设备的互联互通。

**设计目标**：

- 面向消费者：**实现不同的终端设备之间的快速连接、能力互补、资源共享**
- 面向应用开发者：**使得应用程序的开发实现与不同终端设备的形态差异无关**
- 面向设备开发者：**根据设备的资源能力和业务特征进行灵活裁剪**

### 关键特性

| 关键特性 | 描述 |
| --- | --- |
| 编程模型 | 提供基于线程的编程模型，为开发者实现了POSIX和CMSIS-RTOS API（ARM Cortex™的MCU 软件接口标准），便于应用在不同平台的迁移。HarmonyOS采用了组件化的设计方案，支持多种开发语言的API。支持阻塞式I/O 和分阶段I/O。 |
| 任务调度 | 时间片轮转调度。 |
| 内存分配 | 支持动态内存分配。 |
| 软件更新 | 提供了多种软件更新机制。 |
| 网络服务 | 提供了丰富的网络服务。星闪NearLink。 |
| AI支持 | 官方支持，目前兼容TensorFlow Lite，MindSpore Lite等库 |
| 其他 | 多内核设计、分布式软总线技术 |

1. **多内核设计**
   a. 分为**内核层、系统服务层、框架层、应用层**，支持按设备需求**裁剪功能和模块**。

b. 手机端主要采用**Linux内核**；IoT设备采用**LiteOS-A、LiteOS-M**内核

   i. **LiteOS-A**：标准设备、支持有MMU的设备；小体积、低功耗、高性能；大量POSIX，可移植性强；统一驱动框架HDF

   ii. **LiteOS-M**：微设备及迷你设备（无MMU）；主要包括内核最小功能集＋KAL＋可选组件；分为硬件相关（HAL接口）和无关层

2. **分布式软总线技术**：

   a. **分布式软总线**：在**1＋8＋N设备间搭建一条"无形"的总线**，具备**自发现、自组网、高带宽、低时延**特点。全场景设备间可以基于软总线完成设备虚拟化、跨设备服务调用、多屏协同、文件分享等分布式业务。

   b. **分布式数据管理**：基于分布式软总线实现应用程序数据和用户数据的分布式管理。**用户数据不再与单一物理设备绑定，业务逻辑与数据存储分离**，应用跨设备运行时数据无缝衔接，为打造一致、流畅的用户体验创造了基础条件。

   c. **分布式任务调度**：基于分布式软总线、分布式数据管理，构建统一的**分布式服务管理（发现、同步、注册、调用）机制**，支持对跨设备的应用进行远程启动、远程调用、绑定/解绑、迁移等操作，能够根据不同设备的能力、位置、业务运行状态、资源使用情况并结合用户的习惯和意图，选择最合适的设备运行分布式任务。

   d. **分布式设备虚拟化**：分布式设备虚拟化平台可以实现**不同设备的资源融合**、**设备管理、数据处理**，将周边设备作为手机能力的延伸，共同形成一个**超级虚拟终端**。

## AliOS Things

面向 IoT 领域的**轻量级**物联网嵌入式OS，适用于**小型物联网基础设备**，可应用在智能家居、智慧城市等领域。

**主要特点**：开发简易、服务丰富、优异性能、更新便捷

1. **Rhino内核**：基于Rhino的实时OS进行研发的，具有**小FootPrint（内存占用小、两种分配方式）、低功耗（提供了空闲CPU模式）、实时性、多任务运行（基于优先级的抢占式调度和循环调度）等特点**

2. **网络服务**：

   a. **uMesh**: 提供了自组织网络和实现设备间本地互联的能力。 uMesh的特点是自组织、自我修复和多跳，适用于需要大规模部署的场景。

   b. **SAL**: 加速具有不同连接能力的MCU通信芯片的开发

   c. **LwIP**: 深度定制和优化的协议栈，包括IPv4/IPv6、 TCP/UDP、 ICMP、 ARP等

   d. **LoRaWAN**支持：支持LoRaWAN的Class A和Class C两种模式的开发

3. **开发环境**：多种开发语言；强大的IDE；多种Linux工具；组件式开发；云端一体在线开发环境

4. **系统安全**：**系统级别+芯片级别**；支持可信运行环境LinkTEE、可信身份认证Link ID²、对称/非对称密钥，以及基于此的可信连接和安全服务

## ROS2

机器人操作系统

1. **节点**：是基本计算单元

   a. 可分别负责传感器采集、感知、规划或控制

   b. 通过话题、服务和动作协作，形成机器人应用的计算图

   c. 一个进程可包含多个节点，一个节点可包含多个通信接口

2. **三种通信机制**：根据数据交互方式和任务持续时间，选择 Topic、Service 或 Action

| 通信机制 | 交互方式 | 典型场景 | 响应与反馈 |
| --- | --- | --- | --- |
| Topic<br>话题 | 发布/订阅<br>持续传递消息 | 传感器数据<br>图像、里程计 | 无逐条业务响应<br>发布者不等待处理结果 |
| Service<br>服务 | 请求/响应<br>完成一次操作 | 参数查询<br>电机启停、设备复位 | 返回一次执行结果<br>不提供标准过程反馈 |
| Action<br>动作 | 目标/反馈/结果<br>管理长时间任务 | 移动到目标位置<br>机械臂执行运动 | 过程反馈与最终结果<br>可以请求取消目标 |

> Service **也支持异步调用**，是否阻塞取决于客户端 API 和执行方式。

   a. **动作的要素**：目标Goal（客户端请求完成的任务）、反馈Feedback（执行过程中的进度信息）、Result（任务结束后的状态和结果）

   b. **ROS 2 的一个动作由 3 个服务（SendGoal、GetResult、CancelGoal）和 2 个话题（Feedback、Status）组成**

3. **模块化设计与软件复用**

   a. **软件包**组织节点程序、消息接口、参数文件、Launch 文件和依赖信息

   b. 通过**标准消息接口屏蔽硬件差异**，更换驱动后，上层算法通常可以复用。

   c. 替换驱动时需保持**话题名称、消息类型、坐标系、单位和QoS兼容**。

**思考题3**

1. **Contiki OS的protothreads跟传统的多线程有什么区别？**

   a. protothreads**共享一个栈**，采用**协作式**调度，内存开销小，不能使用普通局部变量

   b. 传统多线程通常每个线程有独立栈，**可保存局部变量**和执行现场，支持抢占式或协作式调度，资源开销较大。

2. **Contiki OS中的RPL协议与CTP协议有什么区别？**

   a. CTP主要支持**多对一**的数据收集；

   b. RPL是面向低功耗有损网络的IPv6路由协议，支持多对一、一对多和一对一通信。

3. **RIOT GNRC中哪一层实现了对下层的各类网络协议的一个抽象，使得GNRC可以兼容大量的链路层协议而无需对其代码进行修改**

   最底层的 `gnrc_netdev` **层**，通过抽象下层网络设备，使上层代码兼容不同链路层协议。

4. **RIOT中一个外围设备对应SAUL中哪个数据结构？其传感器数据和控制执行器的参数存在哪个数据结构中？**

   每个外围设备对应 `saul_reg_t`；传感器数据和控制执行器的参数存放在 `phydat_t` 中。

5. **AliOS Things基于哪种实时操作系统内核？其主要特点是什么？**

   基于 **Rhino实时内核**，具有内存占用小、低功耗、实时性和多任务等特点，支持优先级抢占和时间片轮转调度。

6. **AliOS Things中哪个协议实现自组织网络功能？哪个协议实现轻量级的TCP/IP功能？**

   uMesh**实现自组织、自修复和多跳网络**；**LwIP**实现轻量级TCP/IP功能。

7. **HarmonyOS中LiteOS-A、LiteOS-M分别适用哪些场景？**

   a. LiteOS-A**适用于资源较丰富、有MMU的设备，支持虚拟内存映射；**

   b. LiteOS-M适用于资源受限、无MMU的微控制器设备。

8. **分布式软总线含义**：通过**软件**在多个设备之间建立统一的通信通道，具有自发现、自组网、高带宽、低时延等特点，支持跨设备服务调用、多屏协同和文件分享。

# 研究前沿和发展趋势1

1. **高安全性**：Tock

2. **高拓展性**：支持**多种编程语言、多种通信协议**，方便功能扩展 WAIT、TinyNet、**FireStorm**
3. **环境自适应性**：Ink
4. **Agent OS**：AIOS
5. **具身智能OS**：Robonix